Operations Research
Midterm Project

b13701220, b13703048, b13705036, b13303079

May 2026

1 Problem Description and Notation

The IEDO company operates n_S stations, n_C cars across n_L levels, and books n_K orders over an n_D-day horizon. Each order k specifies a requested level ℓ_k, a pickup station/time (s_k^p, t_k^p), a return station/time (s_k^r, t_k^r), and a revenue $R_k = F_{\ell_k} \cdot H_k$. The company decides (i) which orders to accept, (ii) which car serves each accepted order—possibly via a one-step *upgrade* (a level-ℓ order may be served by a level-ℓ or level-$(\ell{+}1)$ car), and (iii) when and where employees relocate cars. Accepting order k earns R_k; rejecting it costs $2R_k$. The objective is to maximize total profit.

Operating rules. After a car is returned, IEDO assumes a 1 h late buffer plus a 3 h cleaning, so the car becomes *ready* $\Theta := 240$ min after the scheduled return time at its return station. A car must be at the pickup station at least $\Delta := 30$ min before pickup. Cars may be relocated between stations i and j in T_{ij} minutes (symmetric, T_{ii}=0, multiple of 30). The sum of relocation minutes over all employee-driven moves is bounded by B. All cars are considered ready at the origin of the planning horizon.

Common notation (used in §2–§5).

Symbol	Meaning
S, C, K	sets of stations, cars, and orders
ℓ_c, s_c^0	level and initial station of car c
ℓ_k, s_k^p, s_k^r, t_k^p, t_k^r, R_k	level, pickup/return station, pickup/return time (min from 2023/01/01 00:00), revenue of order k
T_{ij}	relocation time (min) between stations i and j, symmetric
B	total relocation-time budget (min)
$\Delta = 30,\ \Theta = 240$	pickup-lead time and post-return-ready buffer (min)
$\mathrm{lvl}(c, k)$	predicate $\ell_k \le \ell_c \le \ell_k + 1$ (car c may serve order k, possibly as an upgrade)

2 Problem 1: Mixed-Integer Programming Model

We formulate the planning problem as a per-car path-flow MILP on a time-feasibility DAG. Each car traces an ordered (possibly empty) sequence of orders from a virtual source (its initial station at time 0) to a virtual sink. An arc exists between two orders only if a single car can serve them consecutively under the cleaning, lead-time, and level rules.

2.1 Feasibility-filtered index sets

$$\begin{aligned}
\mathcal{V}^{\text{src}} &= \big\{(c,k) \in C \times K : \text{lvl}(c,k),\ \ T_{s_c^0,\, s_k^p} \le t_k^p - \Delta\big\},\\
\mathcal{V}^{\text{chn}} &= \big\{(c,i,j) \in C \times K \times K : i \ne j,\ \text{lvl}(c,i),\ \text{lvl}(c,j),\ \ t_i^r + \Theta + T_{s_i^r,\, s_j^p} \le t_j^p - \Delta\big\},\\
\mathcal{V}^{\text{snk}} &= \big\{(c,k) \in C \times K : \text{lvl}(c,k)\big\}.
\end{aligned}$$

Restricting variables to these sparse sets dramatically reduces model size and is enough to encode every operational rule: source feasibility encodes the initial-position lead-time constraint; chain feasibility encodes the cleaning, ready-buffer, relocation, and lead-time rules in one inequality.

2.2 Decision variables

- $x^{\text{src}}_{c,k} \in \{0,1\}$ for $(c,k) \in \mathcal{V}^{\text{src}}$: car c starts its tour by serving order k.
- $x^{\text{chn}}_{c,i,j} \in \{0,1\}$ for $(c,i,j) \in \mathcal{V}^{\text{chn}}$: car c serves i then immediately j.
- $x^{\text{snk}}_{c,k} \in \{0,1\}$ for $(c,k) \in \mathcal{V}^{\text{snk}}$: car c ends its tour at order k.
- $y_k \in \{0,1\}$: order k is accepted.

2.3 Constraints

(C1) At most one source per car. A car either sits idle or starts exactly one tour:

$$\sum_{k:\,(c,k)\in\mathcal{V}^{\text{src}}} x^{\text{src}}_{c,k} \;\le\; 1, \qquad \forall c \in C.$$

(C2) Flow conservation at every (car, order) node. For every (c,k) with $\text{lvl}(c,k)$,

$$\underbrace{x^{\text{src}}_{c,k} \cdot \mathbb{1}[(c,k) \in \mathcal{V}^{\text{src}}] + \sum_{i:\,(c,i,k)\in\mathcal{V}^{\text{chn}}} x^{\text{chn}}_{c,i,k}}_{\text{flow into node } (c,k)} \;=\; \underbrace{x^{\text{snk}}_{c,k} + \sum_{j:\,(c,k,j)\in\mathcal{V}^{\text{chn}}} x^{\text{chn}}_{c,k,j}}_{\text{flow out of node } (c,k)}.$$

(C3) Acceptance links flow to y. Each accepted order is served by exactly one car:

$$\sum_{c\in C}\left(x^{\text{src}}_{c,k} \cdot \mathbb{1}[(c,k) \in \mathcal{V}^{\text{src}}] + \sum_{i:\,(c,i,k)\in\mathcal{V}^{\text{chn}}} x^{\text{chn}}_{c,i,k} \right) \;=\; y_k, \qquad \forall k \in K.$$

(C4) Relocation budget. Only positive-distance arcs consume budget:

$$\sum_{(c,k)\in\mathcal{V}^{\text{src}}} T_{s_c^0,\, s_k^p}\, x^{\text{src}}_{c,k} \;+\; \sum_{(c,i,j)\in\mathcal{V}^{\text{chn}}} T_{s_i^r,\, s_j^p}\, x^{\text{chn}}_{c,i,j} \;\le\; B.$$

2.4 Objective

The IEDO profit is

$$\max \sum_{k\in K} R_k\, y_k \;-\; 2\sum_{k\in K} R_k\,(1-y_k) \;\equiv\; 3\sum_{k\in K} R_k\, y_k \;-\; 2\sum_{k\in K} R_k,$$

where the second form drops the additive constant for solver efficiency. The model is a pure binary program. Acyclicity of the chain DAG (times are strictly increasing along any feasible chain) precludes subtours, so no extra cuts are needed.

3 Problem 1: Optimal Plan for Instance 5

Solving the MILP of §2 on `instance05.txt` with Gurobi (Python 3.12, default settings) yields the provably optimal profit **NT$ 106,800** in well under one second. Below we present the plan in operational language.

3.1 Headline

Item	Amount (NT$)
Sales revenue from the 8 accepted orders	117,000
Compensation paid to the 2 rejected customers (2× revenue)	10,200
Net profit	**106,800**

We accept 8 of 10 orders, decline 2, perform 1 free upgrade, and use 1,080 of the 1,200 budgeted relocation minutes (90%). No feasible alternative plan does strictly better.

3.2 Order-by-order decision

Order	Lv	Pickup	Return	Rev.	Decision	Car	Upg.
1	1	St.1, Day4 01:00	St.6, Day4 20:00	1,900	Reject (−3,800)	—	—
2	1	St.4, Day1 16:00	St.5, Day3 00:00	3,200	Reject (−6,400)	—	—
3	1	St.5, Day2 03:30	St.1, Day4 11:30	5,600	Accept	3	No
4	1	St.1, Day3 12:00	St.2, Day5 15:00	5,100	Accept	1	No
5	2	St.6, Day1 00:00	St.6, Day4 20:00	36,800	Accept	6	No
6	1	St.3, Day4 23:00	St.4, Day5 14:00	1,500	Accept	4	**Yes**
7	2	St.2, Day2 03:30	St.1, Day4 23:30	27,200	Accept	5	No
8	2	St.6, Day5 04:30	St.6, Day5 14:30	4,000	Accept	6	No
9	2	St.3, Day1 19:30	St.4, Day4 15:30	27,200	Accept	4	No
10	1	St.4, Day1 06:00	St.2, Day5 06:00	9,600	Accept	2	No

Why decline? Orders 1 and 2 are small-revenue level-1 jobs whose pickup windows collide with strictly more lucrative commitments: serving them would force the cancellation of orders 4/10 or 7/9 (which together earn over NT$ 69,000). Paying NT$ 10,200 in compensation is cheaper than that opportunity cost. *Free upgrade.* Order 6 (level-1) is most profitably satisfied by the level-2 Car 4, which has just completed Order 9 and sits at a convenient station.

3.3 Relocation work orders

Five employee-driven moves, totalling 1,080 of the 1,200 budgeted minutes:

#	Car	From	To	Depart	Arrive	Duration
1	2	St.2	St.4	Day1 01:30	Day1 05:30	240
2	4	St.5	St.3	Day1 15:30	Day1 19:00	210
3	5	St.4	St.2	Day1 23:00	Day2 03:00	240
4	3	St.3	St.5	Day1 23:30	Day2 03:00	210
5	4	St.4	St.3	Day4 19:30	Day4 22:30	180
					Total	**1,080** / 1,200

3.4 Per-car schedules

- **Car 1** (L1, st.1): Order 4 only [pick St.1 Day3 12:00 → ret St.2 Day5 15:00]. No relocation.
- **Car 2** (L1, st.2): Relocate St.2→St.4 (Day1 01:30, 240 min) ⇒ Order 10 [Day1 06:00 → Day5 06:00 St.2].
- **Car 3** (L1, st.3): Relocate St.3→St.5 (Day1 23:30, 210 min) ⇒ Order 3 [Day2 03:30 → Day4 11:30 St.1].
- **Car 4** (L2, st.5): Relocate St.5→St.3 ⇒ Order 9 [Day1 19:30 → Day4 15:30 St.4]; ready Day4 19:30, relocate St.4→St.3 ⇒ Order 6 *(upgrade)* [Day4 23:00 → Day5 14:00].
- **Car 5** (L2, st.4): Relocate St.4→St.2 ⇒ Order 7 [Day2 03:30 → Day4 23:30 St.1].
- **Car 6** (L2, st.6): Order 5 [Day1 00:00 → Day4 20:00 St.6]; ready Day5 00:00 ⇒ Order 8 [Day5 04:30 → Day5 14:30 St.6]. No relocation.

4 Problem 2 & 3: Heuristic Algorithm

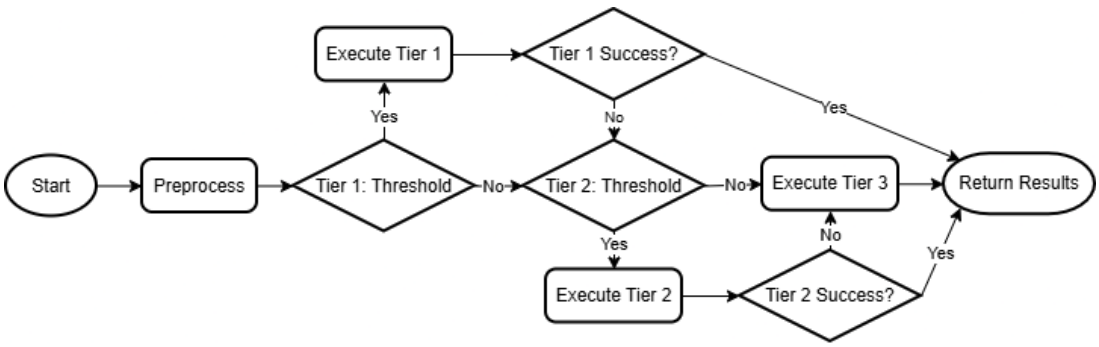
While the MILP of §2 solves Instance 5 in well under one second, on a larger instance (say $n_K = 10{,}000$) the chain-arc count is $\mathcal{O}(n_C \cdot n_K^2) \approx 10^{11}$, well beyond the three-minute budget. We therefore design a three-tier heuristic that recovers the exact MILP at small scales and degrades gracefully.

4.1 Algorithm design

To solve the car-rental scheduling problem across varying scales and ensure the program produces a feasible solution within the 3-minute limit, our algorithm adopts a **Three-Tier Adaptive Architecture**. The key idea is to use different strategies for different instance sizes. Small instances are solved by a Path-flow MIP (the model of §2 with a tight solver time limit), medium-sized instances by a Revenue-first Insertion Heuristic, and large instances by a fast Heap-Based Chronological Greedy. This achieves a good balance between solution quality and computational efficiency.

4.2 Algorithm structure

Flow chart



Data preprocessing

- **Time cache.** Convert all string-formatted times into "minutes elapsed since 2023/01/01 00:00." A dictionary cache avoids repeated string parsing, reducing I/O conversion

cost on large instances.

- **Moving-time matrix.** Build a 2-D array `moving_time[i][j]` so all relocation-time queries are $\mathcal{O}(1)$.

Tier 1: Small instances (Path-flow MIP)

- **Trigger:** $n_K \leq 160$ and $n_C \cdot n_K^2 \leq 700{,}000$.
- **Logic:** Build the path-flow MILP of §2. Decision variables $x_{c,i,j}$ encode whether car c, after serving order i, has enough cleaning and moving time to next serve j. Gurobi's time limit is set to 20 s; if solved successfully, return the optimal schedule directly.

Tier 2: Medium instances (Revenue-first Insertion Heuristic)

- **Trigger:** $n_K \leq 6{,}000$ and $n_C \cdot n_K \leq 600{,}000$.
- **Logic:** Driven by the intuition of "maximize business profit."
  - **a.** Sort all orders in descending order of R_k.
  - **b.** Process high-revenue orders first; search candidate cars satisfying the level rule (same level or free upgrade).
  - **c.** Use binary search to attempt inserting the new order into the car's existing schedule (sorted by pickup time).
  - **d.** Check the immediately previous and next orders at the insertion position to ensure no conflict after accounting for cleaning and moving times. Assign the car that consumes the least additional relocation budget.

Tier 3: Large instances (Heap-Based Chronological Greedy)

- **Trigger:** ultimate fallback when Tiers 1–2 time out or the instance is too large.
- **Logic:** simulate the passage of time in the real world.
  - **a.** Maintain a min-heap for each (station, car-level) pair, keyed by "earliest ready time."
  - **b.** Sort orders chronologically by pickup time.
  - **c.** For each order in order, attempt to pop the earliest-ready car from the pickup station's heap in $\mathcal{O}(\log n_C)$.
  - **d.** If no local car is available, scan a pre-built "nearest-stations" list, evaluate moving time and remaining budget, and relocate a car from a nearby station. After serving, update the ready time and push the car into the return station's heap.

4.3 Time complexity

Let $K = n_K, C = n_C, S = n_S, L = n_L$.

Preprocessing. Building the moving-time matrix is $\mathcal{O}(S^2)$ and converting all order times is $\mathcal{O}(K)$:

$$T_{\text{prep}} = \mathcal{O}(K + C + S^2).$$

Tier 1 MIP. The main arc-creation loop is $\mathcal{O}(n_C \cdot n_K^2)$:

$$T_{\text{MIP}} = \mathcal{O}(n_C \cdot n_K^2).$$

Tier 2 Insertion heuristic.

- Sort orders: $\mathcal{O}(n_K \log n_K)$.

- For each order, scan all candidate cars and binary-search the insertion position: $\mathcal{O}(n_C \log n_K)$.
- Actual insert: $\mathcal{O}(n_K)$ per insertion.
- Total: $T_{\text{insertion}} = \mathcal{O}(n_K \cdot n_C \cdot \log n_K + n_K^2)$.

Tier 3 Heap greedy.

- Precompute nearest-neighbour list: $\mathcal{O}(n_S^2 \log n_S)$.
- Sort orders: $\mathcal{O}(n_K \log n_K)$.
- Process each order: local-station check $\mathcal{O}(n_L)$, cross-station scan $\mathcal{O}(\text{scan_limit} \cdot n_L \cdot \log n_C)$.
- Total (large n_S): $T_{\text{greedy}} = \mathcal{O}(n_S^2 \log n_S + n_K \cdot \sqrt{n_S} \cdot n_L \cdot \log n_C)$.

5 Problem 4: Numerical Experiments

[To be written: factor/scenario design, instance generator, benchmark choice, results table with average and standard deviation of optimality gaps, and a 2×2 histogram across the four scenarios.]

References

[1] T. T. Tran, A. Araujo, and J. C. Beck. Decomposition methods for the parallel machine scheduling problem with setups. *INFORMS Journal on Computing*, 28(1):83–95, 2016.